Our apporach

```
project/
 |- t0_experiments.jl
 |- t1_experiments.jl
 |- source/
```

Two phases

Each gets one notebook
- t0_… till step 5
- t1_… till step 8

Shared code Modularized in Source

Step 1 – The Graph

```
project/
 |- t0_experiments.jl
 |- t1_experiments.jl
 |- source/
     |- Graph.jl
```

Graph module

Based on given implementation.

Can be constructed from the json document.

Step 2 – Calculate r

```
project/
 |- t0_experiments.jl
 |- t1_experiments.jl
 |- source/
     |- Graph.jl
     |- SplitCoefficients.jl
```

Split Coefficients Module

Exports one Function that returns a Dict r

r contains only shortest paths

Step 3 – Model parameters

```
project/
 |- t0_experiments.jl
 |- t1_experiments.jl
 |- source/
     |- Graph.jl
     |- SplitCoefficients.jl
     |- TrafficMatrix.jl
     |- Scenario.jl
```

Traffic Matrix:
Exports one function to parse demands vector from tm json

Scenario:
Exports one function to parse maxSeg out of scenarios json

Step 4 – The model

```
project/
 |- t0_experiments.jl
 |- t1_experiments.jl
 |- source/
     |- Graph.jl
     |- SplitCoefficients.jl
     |- TrafficMatrix.jl
     |- Scenario.jl
     |- Model.jl
```

Model module

Follows the builder pattern to easily use it for both notebooks

Each constraint gets its own builder function

Step 4 – The Constraints

Decision Variables: $x_{i,j}^d \in \{0, 1\}$

- 1 if demand d uses segment (i,j), 0 otherwise.

1. Flow Conservation (Eq. 1): $\sum_j x_{ij}^d - \sum_j x_{ji}^d = \text{rhs}$

- Guarantees path continuity from source s to target t.

2. Segment Limit (Eq. 2): $\sum_{i,j} x_{ij}^d \le \text{maxSeg}$

- Physical hardware limit on router packet headers.

3. Link Load Constraint (Eq. 3): $\text{Load}(a) \le \lambda_{\max} \cdot c(a)$

- Maps segment routing to real physical cables using ECMP coefficients $r(i,j,a)$.

(Pseudo) Code from the functions

SetFlowConservation
SetSegmentCap

Step 4 – Lex descend

Core Objective (Eq. 5): Minimize Maximum Link Utilization ($\text{MLU} / \lambda_{\max}$).

The Problem: Minimizing only $\lambda_{\max}$ leaves other links unoptimized and close to saturation.

Our Solution — Lexicographic Descent (OWA Gadget):

- Level 1: Solve for absolute MLU (λ^*).

- Deeper Levels ($k \ge 2$): Freeze previous load and minimize the sum of the k most loaded links (S_k).

Key Implementation Detail: Uses continuous slacks $d(a)$ and threshold t_k $\implies$ Zero extra binary variables added!

(Pseudo) Code from the

Objective

Lex descend

Step 5 – T0 Experiments

Slides for results and problems

Step 6 – Extensions for t1

```
project/
 |- t0_experiments.jl
 |- t1_experiments.jl
 |- source/
      |- Graph.jl
      |- SplitCoefficients.jl
      |- TrafficMatrix.jl
      |- Scenario.jl
      |- Model.jl
```

New functions for Scenario
- Parse downtime arcs
- Parse the budget

New function for Graph
- Build graph with downtime

Step 7 – Updated Model

Maintenance Context ($t=1$): Links go down ($G_1 = V \setminus q(1)$) $\implies$ Network needs rerouting.

Operational Constraint: Reconfiguring routers causes human costs and QoS risks.

Budget Constraint (Eq. 4): $\text{dist}(P^0, P^1) \le \kappa(1)$

- Limits the number of waypoint modifications between nominal ($t=0$) and maintenance ($t=1$) plans.

Implementation: Linearized absolute differences $\vert{}x^{d,1} - x^{d,0}\vert{}$ using auxiliary variables segmentChange in add_budgetBounds!.

(Pseudo) Code for bounds from set_loadBounds function

Step 8 – T1 experiments

Several slides of results

f